

Laporan Praktikum

Kecerdasan Komputasional (Bayesian Network and Inference)

Identitas Praktikan

Nama	NRP	Kelas	Jurusan
Wayan Raditya Putra	5054241029	N	Rekayasa Kecerdasan Artifisial

Dosen Pengampu

Nama Dosen
Imam Mustafa Kamal, S.ST, Ph.D

Judul Praktikum

Inferensi Probabilistik Menggunakan Bayesian Network dan Enumerasi

Tujuan

Mempelajari bagaimana model **Bayesian Network** digunakan untuk merepresentasikan hubungan ketergantungan antar variabel acak, serta memahami cara melakukan **inferensi probabilistik** menggunakan metode **enumerasi**. Praktikum ini bertujuan agar mahasiswa dapat menghitung distribusi posterior berdasarkan bukti yang diberikan, dengan mengimplementasikan proses inferensi pada jaringan Bayesian menggunakan pendekatan komputasional.

Inisialisasi dan Import Library

Pada bagian ini dilakukan proses *import* terhadap beberapa modul yang digunakan dalam praktikum.

Baris kode berikut:

```
In [2]: from probability import *
        from utils import print_table
        from notebook import psource, pseudocode, heatmap
```

```
d:\AI_Journey\Computational_Intelligence\CI\Lib\site-packages\qpsolvers\solvers\_
_init__.py:880: UserWarning: no QP solver found on your system, you can install s
olvers from PyPI by ``pip install qpsolvers[open_source_solvers]``
  warnings.warn(
```

Bayesian Networks

Bayesian Network merupakan representasi grafis dari **distribusi probabilitas gabungan (joint probability distribution)** yang menyandikan sekumpulan hubungan ketergantungan dan independensi bersyarat antar variabel acak. Setiap node dalam jaringan ini mewakili satu variabel acak, sedangkan arah panah (edge) menunjukkan hubungan sebab-akibat atau ketergantungan langsung di antara variabel-variabel tersebut. Dengan menggunakan struktur graf berarah tanpa siklus (*Directed Acyclic Graph* atau DAG), Bayesian Network memungkinkan kita untuk memodelkan hubungan sebab-akibat yang kompleks secara efisien dan terstruktur.

Secara matematis, distribusi probabilitas gabungan dari semua variabel dalam jaringan dapat dituliskan sebagai hasil perkalian dari probabilitas setiap variabel terhadap orang tuanya, yaitu:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

Artinya, untuk menghitung probabilitas gabungan dari semua variabel dalam sistem, kita cukup mengalikan probabilitas bersyarat dari setiap variabel terhadap *parent*-nya. Dengan cara ini, Bayesian Network secara signifikan mengurangi kompleksitas komputasi dibandingkan dengan representasi *full joint distribution* tradisional.

Pada implementasi di praktikum ini, jaringan Bayesian direpresentasikan menggunakan kelas **BayesNet**, sedangkan setiap simpul (node) direpresentasikan oleh kelas **BayesNode**. Kelas **BayesNet** berfungsi sebagai wadah yang menyimpan seluruh node dan mengatur hubungan ketergantungan antarvariabel, sementara **BayesNode** merepresentasikan satu variabel acak beserta **Conditional Probability Table (CPT)**-nya. Tabel CPT berisi probabilitas bersyarat dari variabel tersebut terhadap parent-nya, sesuai dengan rumus **$P(X \mid \text{Parents})$** .

Implementasi yang digunakan dalam praktikum ini berfokus pada **variabel boolean**, di mana setiap node hanya memiliki dua kemungkinan nilai: *True* atau *False*. Dengan pendekatan ini, perhitungan probabilitas menjadi lebih sederhana dan mudah divisualisasikan, terutama untuk memahami konsep dasar inferensi probabilistik.

Secara umum, **BayesNode** memiliki tiga komponen utama:

1. **Nama variabel (X)** – merepresentasikan variabel acak yang diwakili oleh node.
2. **Daftar parent** – menyimpan node-node lain yang menjadi penyebab langsung dari node tersebut.
3. **Conditional Probability Table (CPT)** – mendefinisikan peluang variabel X bernilai *True* untuk setiap kombinasi nilai parent-nya.

Melalui struktur ini, setiap node dalam Bayesian Network dapat berinteraksi dengan node lain dalam bentuk hubungan sebab-akibat yang dinyatakan secara probabilistik. Bagian berikut akan membahas lebih dalam mengenai implementasi kelas **BayesNode** yang menjadi dasar pembentukan jaringan Bayesian dalam sistem inferensi ini.

```
In [3]: psource(BayesNode)
```

class BayesNode:

*"""A conditional probability distribution for a boolean variable,
P(X | parents). Part of a BayesNet."""*

def __init__(self, X, parents, cpt):

*"""X is a variable name, and parents a sequence of variable
names or a space-separated string. cpt, the conditional
probability table, takes one of these forms:*

** A number, the unconditional probability P(X=true). You can
use this form when there are no parents.*

** A dict {v: p, ...}, the conditional probability distribution
P(X=true | parent=v) = p. When there's just one parent.*

** A dict {(v1, v2, ...): p, ...}, the distribution P(X=true |
parent1=v1, parent2=v2, ...) = p. Each key must have as many
values as there are parents. You can use this form always;
the first two are just conveniences.*

*In all cases the probability of X being false is left implicit,
since it follows from P(X=true).*

```
>>> X = BayesNode('X', '', 0.2)
>>> Y = BayesNode('Y', 'P', {T: 0.2, F: 0.7})
>>> Z = BayesNode('Z', 'P Q',
...   {(T, T): 0.2, (T, F): 0.3, (F, T): 0.5, (F, F): 0.7})
"""
```

if isinstance(parents, str):

parents = parents.split()

We store the table always in the third form above.

if isinstance(cpt, (float, int)): *# no parents, 0-tuple*

cpt = {}: cpt

elif isinstance(cpt, dict):

one parent, 1-tuple

if cpt **and** isinstance(list(cpt.keys())[0], bool):

cpt = {(v,): p **for** v, p **in** cpt.items() }

assert isinstance(cpt, dict)

for vs, p **in** cpt.items():

assert isinstance(vs, tuple) **and** len(vs) == len(parents)

assert all(isinstance(v, bool) **for** v **in** vs)

assert 0 <= p <= 1

```

self.variable = X
self.parents = parents
self.cpt = cpt
self.children = []

```

```

def p(self, value, event):
    """Return the conditional probability
     $P(X=value \mid \text{parents}=\text{parent\_values})$ , where parent_values
    are the values of parents in event. (event must assign each
    parent a value.)
    >>> bn = BayesNode('X', 'Burglary', {T: 0.2, F: 0.625})
    >>> bn.p(False, {'Burglary': False, 'Earthquake': True})
    0.375"""
    assert isinstance(value, bool)
    ptrue = self.cpt[event_values(event, self.parents)]
    return ptrue if value else 1 - ptrue

```

```

def sample(self, event):
    """Sample from the distribution for this variable conditioned
    on event's values for parent_variables. That is, return True/False
    at random according with the conditional probability given the
    parents."""
    return probability(self.p(True, event))

```

```

def __repr__(self):
    return repr((self.variable, '.'.join(self.parents)))

```

Kelas BayesNode

Kelas `BayesNode` merepresentasikan **distribusi probabilitas bersyarat (Conditional Probability Distribution)** untuk sebuah variabel boolean dalam jaringan Bayesian, dengan bentuk umum **$P(X \mid \text{Parents})$** . Kelas ini merupakan komponen dasar dari struktur **BayesNet**, di mana setiap node menggambarkan satu variabel acak beserta hubungan probabilitiknya terhadap variabel-variabel penyebab (parent).

Konstruktor `__init__(self, X, parents, cpt)` berfungsi untuk menginisialisasi node dengan tiga parameter utama:

1. **X** — nama variabel yang diwakili oleh node.
2. **parents** — daftar variabel parent, yang bisa diberikan dalam bentuk string (misalnya `"Burglary Earthquake"`) atau list. Jika diberikan sebagai string, parameter ini akan dipisahkan menjadi list menggunakan metode `split()`.
3. **cpt (Conditional Probability Table)** — tabel probabilitas bersyarat yang menyimpan nilai **$P(X = \text{True} \mid \text{Parents})$** . Tabel ini dapat diberikan dalam tiga bentuk:

- **Tanpa parent:** berupa satu angka tunggal (misalnya `0.2`), yang berarti **$P(X=True)$** tanpa kondisi apapun.
- **Satu parent:** berupa dictionary `{True: p1, False: p2}` yang menunjukkan probabilitas $X=True$ untuk masing-masing nilai parent.
- **Lebih dari satu parent:** berupa dictionary dengan tuple sebagai kunci, misalnya `{(True, True): 0.3, (True, False): 0.2, (False, True): 0.5, (False, False): 0.7}`.

Apapun bentuk input awalnya, konstruktor akan menyimpannya dalam format standar (bentuk ketiga), sehingga setiap key berupa tuple dengan panjang yang sama dengan jumlah parent. Proses ini memastikan bahwa seluruh nilai valid secara probabilistik (antara 0 dan 1) serta memiliki struktur data yang seragam untuk inferensi selanjutnya.

Setelah inisialisasi, setiap objek `BayesNode` memiliki atribut:

- `self.variable` → menyimpan nama variabel.
- `self.parents` → menyimpan daftar variabel parent.
- `self.cpt` → berisi tabel probabilitas bersyarat.
- `self.children` → menyimpan daftar node turunan (anak) yang memiliki node ini sebagai parent, berguna untuk membangun relasi jaringan Bayesian secara hierarkis.

Metode `p(self, value, event)` digunakan untuk menghitung **$P(X=value \mid \text{Parents}=\text{parent_values})$** , yaitu probabilitas variabel bernilai `True` atau `False` berdasarkan nilai parent yang diberikan melalui parameter `event`. Nilai `event` berbentuk dictionary, misalnya `{'Burglary': True, 'Earthquake': False}`. Fungsi ini mengambil probabilitas dari tabel CPT berdasarkan kombinasi nilai parent, lalu jika `value=False`, fungsi mengembalikan $1 - P(X=True)$.

Metode `sample(self, event)` digunakan untuk menghasilkan sampel acak (`True` atau `False`) dari distribusi probabilitas bersyarat berdasarkan nilai parent yang diberikan. Proses ini mensimulasikan perilaku acak sistem sesuai probabilitas yang telah ditentukan dalam CPT.

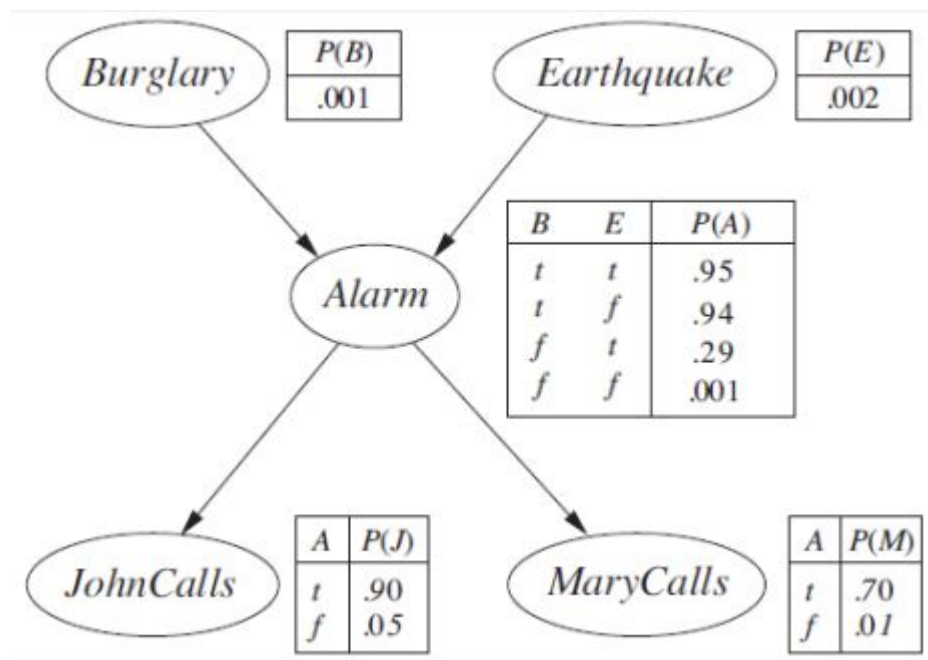
Terakhir, metode `__repr__(self)` memberikan representasi ringkas dari node dalam format `(variable, parents)`, misalnya `('Alarm', 'Burglary Earthquake')`, untuk memudahkan identifikasi struktur jaringan.

Secara keseluruhan, kelas `BayesNode` berperan penting dalam membentuk struktur dasar **Bayesian Network** karena menyimpan seluruh informasi lokal tentang hubungan probabilistik antara variabel dan penyebabnya. Dari kelas inilah nantinya proses **inferensi probabilistik** (seperti enumerasi atau sampling) dapat dilakukan dengan efisien.

Konstruktor pada kelas `BayesNode` menerima tiga parameter utama, yaitu **variable**, **parents**, dan **cpt**. Parameter **variable** menunjukkan nama variabel acak, misalnya `'Burglary'`, `'Earthquake'`, atau `'Alarm'`. Parameter **parents** berisi daftar nama variabel penyebab langsung (*parent nodes*) dari variabel tersebut, dan dapat diberikan dalam bentuk list seperti `['Burglary', 'Earthquake']` atau dalam bentuk string yang dipisahkan dengan spasi, misalnya `'Burglary Earthquake'`.

Parameter ketiga, **cpt (Conditional Probability Table)**, adalah tabel probabilitas bersyarat yang menyimpan nilai **$P(X=\text{True} \mid \text{Parents})$** untuk setiap kombinasi nilai dari variabel parent. Struktur tabel ini berupa dictionary dengan format `{(v1, v2, ...): p, ...}`, di mana setiap key adalah kombinasi nilai boolean dari parent, dan value adalah probabilitas variabel **X** bernilai *True* pada kondisi tersebut. Jumlah dan urutan nilai dalam tuple key harus sama dengan jumlah dan urutan nama parent yang diberikan. Probabilitas **X=False** tidak perlu dimasukkan, karena secara otomatis dapat dihitung sebagai: $P(X=\text{False} \mid \text{Parents}) = 1 - P(X=\text{True} \mid \text{Parents})$

Contoh implementasi ini dapat dilihat pada **Gambar 14.3** dari buku *Artificial Intelligence: A Modern Approach (AIMA)* yang memperlihatkan jaringan Bayesian dengan lima variabel: **Burglary**, **Earthquake**, **Alarm**, **JohnCalls**, dan **MaryCalls**.



Struktur jaringan tersebut menunjukkan bahwa kejadian **Burglary** dan **Earthquake** memengaruhi probabilitas terjadinya **Alarm**, sedangkan **Alarm** menjadi penyebab bagi variabel **JohnCalls** dan **MaryCalls**. Setiap node memiliki **Conditional Probability Table (CPT)** yang menyatakan hubungan probabilistik dengan parent-nya.

Sebagai contoh, node **Alarm** yang memiliki dua parent (**Burglary** dan **Earthquake**) dapat didefinisikan sebagai berikut:

```
In [4]: alarm_node = BayesNode('Alarm', ['Burglary', 'Earthquake'],
                                {(True, True): 0.95, (True, False): 0.94, (False, True): 0.29, (False, False): 0.001})
```

Kode di atas menunjukkan bahwa probabilitas **Alarm=True** akan tinggi ketika baik **Burglary** maupun **Earthquake** bernilai *True* (0.95), dan sangat rendah (0.001) ketika keduanya *False*. Dengan cara ini, jaringan Bayesian mampu menggambarkan hubungan sebab-akibat antar kejadian secara terukur dan realistis menggunakan model probabilistik.

Dalam implementasi **BayesNode**, jika sebuah node hanya memiliki **satu parent**, maka kita tidak wajib menggunakan tuple untuk mendefinisikan key dalam **Conditional**

Probability Table (CPT). Dengan kata lain, struktur dictionary dapat ditulis secara lebih sederhana tanpa tanda kurung tuple.

Sebagai contoh, pada jaringan Bayesian di Gambar 14.3, node **JohnCalls** dan **MaryCalls** masing-masing hanya memiliki satu parent, yaitu **Alarm**. Karena itu, CPT untuk kedua node ini dapat ditulis menggunakan dua format berbeda yang secara fungsional sama.

Berikut contoh definisinya:

```
In [5]: john_node = BayesNode('JohnCalls', ['Alarm'], {True: 0.90, False: 0.05})
mary_node = BayesNode('MaryCalls', 'Alarm', {(True,): 0.70, (False,): 0.01}) #
# Equivalent to john_node definition.
```

Kedua pendekatan di atas setara karena pada kasus dengan satu parent, kelas `BayesNode` akan secara otomatis mengonversi bentuk `{True: p1, False: p2}` menjadi bentuk standar `{(True,): p1, (False,): p2}`. Dengan demikian, baik menggunakan list (`['Alarm']`) maupun string (`'Alarm'`) untuk parent akan menghasilkan hasil yang sama.

Pada contoh di atas, interpretasinya adalah sebagai berikut:

- Untuk node **JohnCalls**, jika **Alarm=True**, maka probabilitas John menelepon adalah 0.90, dan jika **Alarm=False**, maka probabilitasnya hanya 0.05.
- Untuk node **MaryCalls**, probabilitas Mary menelepon adalah 0.70 ketika **Alarm=True**, dan hanya 0.01 ketika **Alarm=False**.

Fleksibilitas format ini memudahkan penulisan kode, terutama pada kasus di mana sebuah node hanya memiliki satu parent, tanpa perlu menggunakan tuple eksplisit.

Format umum yang digunakan untuk mendefinisikan node seperti **Alarm** juga berlaku untuk semua node lain dalam jaringan Bayesian. Namun, untuk node yang **tidak memiliki parent**, format penulisannya dapat dibuat lebih sederhana lagi. Dalam kasus ini, **Conditional Probability Table (CPT)** hanya perlu menyimpan satu nilai probabilitas tunggal, yaitu **P(X=True)**, karena tidak ada variabel lain yang menjadi penyebab atau kondisi bagi node tersebut.

Sebagai contoh, pada jaringan Bayesian di Gambar 14.3, node **Burglary** dan **Earthquake** merupakan variabel bebas yang tidak memiliki parent. Oleh karena itu, keduanya dapat didefinisikan cukup dengan memberikan nilai probabilitas tunggal sebagai berikut:

```
In [6]: burglary_node = BayesNode('Burglary', '', 0.001)
earthquake_node = BayesNode('Earthquake', '', 0.002)
```

Pada contoh di atas:

- Node **Burglary** memiliki probabilitas dasar sebesar 0.001, yang berarti peluang terjadinya pencurian (*burglary*) hanya 0.1%.
- Node **Earthquake** memiliki probabilitas dasar sebesar 0.002, atau 0.2%, yang menunjukkan kemungkinan terjadinya gempa bumi tanpa dipengaruhi variabel lain.

Karena tidak ada parent, nilai **P(X=False)** tidak perlu dicantumkan secara eksplisit, karena dapat diperoleh dari rumus:

$$P(X = False) = 1 - P(X = True)$$

Dengan demikian, format ini merupakan bentuk paling sederhana dari *Bayesian node*, yang tetap konsisten dengan struktur umum **P(X | Parents)**, hanya saja dalam kasus ini, daftar parent kosong.

Kelas `BayesNode` juga menyediakan metode **p()** yang berfungsi untuk melakukan *lookup* terhadap nilai probabilitas bersyarat dari sebuah variabel berdasarkan kondisi parent-nya. Metode ini memungkinkan kita untuk langsung menghitung nilai **P(X=value | parents=parent_values)** tanpa perlu mengakses tabel probabilitas secara manual.

Metode **p(value, event)** menerima dua argumen:

1. **value** → nilai dari variabel yang ingin dihitung probabilitasnya, yaitu *True* atau *False*.
2. **event** → sebuah dictionary yang berisi pasangan `{variable: value}` yang mewakili nilai dari parent yang relevan.

Fungsi ini kemudian mencari kombinasi nilai parent yang sesuai di dalam **Conditional Probability Table (CPT)** untuk mengembalikan nilai **P(X=True | Parents)**. Jika argumen `value` adalah *False*, maka fungsi otomatis mengembalikan nilai komplementnya, yaitu `1 - P(X=True | Parents)`.

Sebagai contoh, untuk node **JohnCalls**, kita dapat menggunakan metode ini untuk mencari probabilitas John *tidak* menelepon ketika alarm berbunyi dengan kode berikut:

```
In [7]: john_node.p(False, {'Alarm': True, 'Burglary': True}) # P(JohnCalls=False | Alarm=True, Burglary=True)
Out[7]: 0.09999999999999998
```

Hasil dari pemanggilan fungsi tersebut adalah:

$$P(JohnCalls = False | Alarm = True) = 1 - P(JohnCalls = True | Alarm = True)$$

Berdasarkan tabel probabilitas node **JohnCalls**, kita tahu bahwa

$$P(JohnCalls = True | Alarm = True) = 0.90$$

Maka hasil perhitungannya adalah:

$$P(JohnCalls = False | Alarm = True) = 1 - 0.90 = 0.10$$

Namun, Python menggunakan representasi biner untuk bilangan desimal di memori, sehingga nilai 0.1 tidak dapat disimpan secara presisi sempurna. Akibatnya, hasil perhitungan ditampilkan sebagai 0.09999999999999998.

Dalam konteks probabilitas, perbedaan ini tidak signifikan dan dapat dibulatkan menjadi 0.1, sesuai dengan hasil matematis yang benar.

Dengan demikian, metode **p()** memberikan cara yang praktis untuk mengambil nilai probabilitas bersyarat sesuai dengan kondisi yang diberikan, tanpa harus menelusuri CPT secara manual.

Kelas BayesNet

Kelas `BayesNet` digunakan untuk membangun dan merepresentasikan sebuah **Bayesian Network** yang terdiri dari sekumpulan node dengan variabel boolean. Setiap node di dalam jaringan merepresentasikan satu variabel acak beserta relasinya terhadap variabel parent yang menjadi penyebabnya.

```
In [8]: psource(BayesNet)
```

class BayesNet:

"""Bayesian network containing only boolean-variable nodes."""

def **__init__**(self, node_specs=**None**):

"""Nodes must be ordered with parents before children."""

self.nodes = []

self.variables = []

node_specs = node_specs **or** []

for node_spec **in** node_specs:

self.add(node_spec)

def add(self, node_spec):

"""Add a node to the net. Its parents must already be in the net, and its variable must not."""

node = BayesNode(*node_spec)

assert node.variable **not in** self.variables

assert all((parent **in** self.variables) **for** parent **in** node.parents)

self.nodes.append(node)

self.variables.append(node.variable)

for parent **in** node.parents:

self.variable_node(parent).children.append(node)

def variable_node(self, var):

"""Return the node for the variable named var.

>>> burglary.variable_node('Burglary').variable

'Burglary'"""

for n **in** self.nodes:

if n.variable == var:

return n

raise **Exception**("No such variable: {}".format(var))

def variable_values(self, var):

"""Return the domain of var."""

return [True, False]

def **__repr__**(self):

return 'BayesNet({0!r})'.format(self.nodes)

Konstruktor `__init__(self, node_specs=None)` digunakan untuk menginisialisasi jaringan Bayesian berdasarkan daftar spesifikasi node yang disebut **node_specs**. Parameter ini berupa list yang berisi tuple-tuple dengan format: [(X, \text{parents}, \text{cpt})] di mana:

- **X** adalah nama variabel,
- **parents** merupakan daftar atau string nama parent dari variabel tersebut, dan

- **cpt** adalah *Conditional Probability Table* yang berisi nilai **P(X=True | Parents)**.

Urutan elemen dalam **node_specs** sangat penting karena setiap node yang ditambahkan harus didefinisikan setelah semua parent-nya terlebih dahulu. Dengan kata lain, node parent harus muncul sebelum node child agar dependensi probabilitiknya valid.

Konstruktor akan memanggil metode `add()` untuk setiap entri dalam daftar **node_specs**. Fungsi `add()` membuat objek `BayesNode` baru dengan parameter yang diberikan, kemudian menambahkannya ke dalam jaringan. Fungsi ini juga memastikan bahwa:

1. Variabel node belum pernah ditambahkan sebelumnya.
2. Semua parent dari node sudah ada di jaringan.
3. Node baru dimasukkan ke dalam daftar `self.nodes` dan nama variabelnya ke dalam `self.variables`.

Selain itu, setiap kali node baru ditambahkan, referensi ke node parent diperbarui agar menyertakan anak (*child*) baru dalam atribut `children`, sehingga membentuk struktur hierarki antarvariabel.

Metode penting lain dalam kelas `BayesNet` antara lain:

- `variable_node(var)` digunakan untuk mengambil node berdasarkan nama variabelnya. Jika variabel tidak ditemukan, fungsi akan memunculkan exception.
- `variable_values(var)` mengembalikan domain nilai dari variabel (selalu `[True, False]` karena seluruh variabel bersifat boolean).
- `__repr__(self)` memberikan representasi string dari seluruh jaringan dalam format `BayesNet([...])`, yang berguna untuk debugging.

Dengan struktur ini, kelas `BayesNet` dapat membentuk jaringan Bayesian secara otomatis dari daftar spesifikasi node yang terurut. Hal ini memastikan hubungan antarvariabel tetap konsisten dan siap digunakan untuk proses **inferensi probabilistik** di tahap berikutnya.

Konstruktor dari **BayesNet** mengambil setiap item dalam **node_specs** dan menambahkan **BayesNode** ke dalam variabel objek **nodes** dengan memanggil metode `add()`. Metode `add()` akan menambahkan node baru ke jaringan hanya jika seluruh parent-nya telah ada sebelumnya dan variabel yang bersangkutan belum pernah digunakan. Dengan mekanisme ini, jaringan dapat dibangun secara bertahap sesuai dependensi antarvariabel.

Contoh implementasi jaringan Bayesian dari Gambar 14.3 buku *Artificial Intelligence: A Modern Approach (AIMA)* dapat dilihat pada kode berikut:

T, F = True, False

```
burglary = BayesNet([
    ('Burglary', '', 0.001),
    ('Earthquake', '', 0.002),
    ('Alarm', 'Burglary Earthquake',
```

```

        {(T, T): 0.95, (T, F): 0.94, (F, T): 0.29, (F, F): 0.001}),
        ('JohnCalls', 'Alarm', {T: 0.90, F: 0.05}),
        ('MaryCalls', 'Alarm', {T: 0.70, F: 0.01})
    ])

```

Kode di atas mendefinisikan jaringan Bayesian sederhana dengan lima variabel boolean: **Burglary**, **Earthquake**, **Alarm**, **JohnCalls**, dan **MaryCalls**. Dalam jaringan ini, **Burglary** dan **Earthquake** menjadi penyebab langsung terjadinya **Alarm**, sementara **Alarm** memengaruhi probabilitas **JohnCalls** dan **MaryCalls**.

Objek global `burglary` pada contoh di atas merupakan instance dari kelas `BayesNet` yang sepenuhnya dibangun berdasarkan spesifikasi tersebut. Proses ini menunjukkan bagaimana metode `add()` secara otomatis menumbuhkan struktur jaringan yang saling terhubung dan siap digunakan untuk inferensi probabilistik.

```
In [9]: burglary
```

```
Out[9]: BayesNet([('Burglary', ''), ('Earthquake', ''), ('Alarm', 'Burglary Earthquake'), ('JohnCalls', 'Alarm'), ('MaryCalls', 'Alarm')])
```

Metode `variable_node()` pada kelas `BayesNet` memungkinkan kita untuk mengakses langsung objek `BayesNode` tertentu di dalam sebuah jaringan Bayesian berdasarkan nama variabelnya. Dengan cara ini, kita dapat melihat atau bahkan memodifikasi atribut dari node tersebut, termasuk **Conditional Probability Table (CPT)** yang menyimpan nilai probabilitas bersyarat.

Sebagai contoh, perintah berikut digunakan untuk mengambil node dengan variabel `'Alarm'` dari jaringan `burglary`:

```
In [10]: type(burglary.variable_node('Alarm'))
```

```
Out[10]: probability.BayesNode
```

Hasilnya menunjukkan bahwa objek yang dikembalikan adalah instance dari kelas **BayesNode**. Ini menandakan bahwa node `'Alarm'` di dalam jaringan `burglary` memang merupakan node probabilistik yang memiliki struktur lengkap — termasuk daftar parent, anak (children), serta tabel probabilitas bersyarat.

Selanjutnya, kita juga dapat langsung melihat isi dari **Conditional Probability Table (CPT)** milik node tersebut dengan perintah:

```
In [11]: burglary.variable_node('Alarm').cpt
```

```
Out[11]: {(True, True): 0.95,
          (True, False): 0.94,
          (False, True): 0.29,
          (False, False): 0.001}
```

Output yang ditampilkan berupa dictionary yang berisi nilai probabilitas **P(Alarm=True | Burglary, Earthquake)** untuk setiap kombinasi nilai boolean parent-nya:

```

{(True, True): 0.95,
 (True, False): 0.94,

```

```
(False, True): 0.29,  
(False, False): 0.001}
```

Dari hasil tersebut, kita dapat memahami bahwa:

- Jika **Burglary=True** dan **Earthquake=True**, maka probabilitas **Alarm=True** adalah 0.95.
- Jika **Burglary=False** dan **Earthquake=False**, maka probabilitas **Alarm=True** hanya 0.001.

Dengan demikian, metode **variable_node()** tidak hanya berguna untuk navigasi di dalam jaringan Bayesian, tetapi juga memberikan fleksibilitas untuk memeriksa atau memperbarui nilai probabilitas bersyarat (CPT) dari node tertentu sesuai kebutuhan analisis atau eksperimen.

Exact Inference in Bayesian Networks

Bayesian Network merupakan representasi yang lebih ringkas dari **full joint distribution**, namun tetap memiliki kemampuan yang sama dalam melakukan **inferensi probabilistik**, yaitu menjawab pertanyaan tentang distribusi peluang dari variabel acak berdasarkan bukti (*evidence*) yang diketahui.

Dengan kata lain, Bayesian Network memungkinkan kita menghitung probabilitas bersyarat seperti: $P(X \mid E = e)$ di mana **X** adalah variabel yang ingin diketahui, dan **E=e** merupakan kumpulan variabel yang diketahui nilainya (bukti).

Namun, perlu dicatat bahwa algoritma **exact inference** seperti enumerasi ini tidak bersifat *scalable* untuk jaringan yang sangat besar. Jumlah kombinasi kemungkinan nilai variabel akan bertambah secara eksponensial terhadap jumlah node dalam jaringan, sehingga kompleksitas komputasinya menjadi sangat tinggi. Untuk jaringan berukuran besar, biasanya digunakan pendekatan **approximate inference** yang akan dijelaskan pada bagian berikutnya.

Inference by Enumeration

Inferensi dengan enumerasi merupakan metode dasar dalam melakukan perhitungan probabilitas bersyarat secara eksak (*exact*). Teknik ini menggunakan prinsip yang sama seperti fungsi **enumerate_joint_ask** dan **enumerate_joint** pada distribusi gabungan penuh, namun diterapkan dalam konteks Bayesian Network.

Pada implementasi ini, digunakan dua fungsi utama:

- **enumeration_ask(X, e, bn)** → Menghitung distribusi probabilitas bersyarat untuk variabel **X**, diberikan evidence **e**, pada jaringan Bayesian **bn**.
- **enumerate_all(vars, e, bn)** → Melakukan enumerasi atau penjumlahan terhadap semua kombinasi nilai variabel yang belum diketahui, untuk menghitung probabilitas total berdasarkan bukti yang tersedia.

Kedua fungsi tersebut mengimplementasikan algoritma inferensi berbasis enumerasi sebagaimana dijelaskan dalam **Figure 14.9** buku *Artificial Intelligence: A Modern Approach (AIMA)*.

Prosesnya melibatkan langkah-langkah berikut:

1. Mengalikan semua probabilitas bersyarat dari variabel-variabel yang diketahui (evidence).
2. Melakukan penjumlahan terhadap semua kemungkinan nilai variabel tersembunyi (hidden variables) untuk menghitung probabilitas total.
3. Menormalisasi hasil akhir agar distribusi probabilitasnya valid (jumlah total = 1).

Metode ini memberikan hasil inferensi yang akurat dan menjadi dasar bagi banyak algoritma probabilistik lainnya, meskipun dari sisi efisiensi masih terbatas untuk jaringan dengan jumlah node yang besar.

```
In [12]: psource(enumerate_all)
```

```
def enumerate_all(variables, e, bn):
    """Return the sum of those entries in P(variables | e{others})
    consistent with e, where P is the joint distribution represented
    by bn, and e{others} means e restricted to bn's other variables
    (the ones other than variables). Parents must precede children in variables."""
    if not variables:
        return 1.0
    Y, rest = variables[0], variables[1:]
    Ynode = bn.variable_node(Y)
    if Y in e:
        return Ynode.p(e[Y], e) * enumerate_all(rest, e, bn)
    else:
        return sum(Ynode.p(y, e) * enumerate_all(rest, extend(e, Y, y), bn)
                  for y in bn.variable_values(Y))
```

Fungsi **enumerate_all(variables, e, bn)** merupakan bagian inti dari proses inferensi berbasis enumerasi pada jaringan Bayesian. Fungsinya adalah menghitung total probabilitas dari semua kombinasi nilai variabel yang konsisten dengan bukti (evidence) yang diberikan. Dengan kata lain, fungsi ini mengimplementasikan proses penjumlahan terhadap seluruh kemungkinan nilai variabel tersembunyi (hidden variables) untuk mendapatkan probabilitas total dari distribusi gabungan jaringan.

Parameter fungsinya terdiri atas:

- **variables** → urutan variabel dalam jaringan Bayesian. Parent selalu muncul sebelum child.
- **e** → dictionary berisi bukti (evidence), dalam format `{variable: value}`.
- **bn** → objek jaringan Bayesian (`BayesNet`) yang digunakan sebagai representasi distribusi probabilitas.

Langkah-langkah kerja fungsi ini adalah sebagai berikut:

1. **Kasus dasar:**

Jika daftar variabel kosong (`if not variables:`), fungsi mengembalikan `1.0` . Ini adalah kondisi berhenti dari rekursi, karena tidak ada lagi variabel yang perlu dihitung.

2. **Ambil variabel pertama:**

Variabel pertama `Y` diambil dari daftar `variables` , dan sisanya disimpan di `rest` . Node yang sesuai dengan variabel `Y` diambil dari jaringan Bayesian dengan `bn.variable_node(Y)` .

3. **Kasus ketika nilai variabel diketahui (ada dalam evidence):**

Jika `Y` terdapat dalam evidence `e` , maka probabilitas bersyaratnya dihitung menggunakan `Ynode.p(e[Y], e)` dan dikalikan dengan hasil rekursi pada variabel berikutnya:

$$P(Y = e[Y] \mid Parents(Y)) \times \text{enumerate_all}(\text{rest}, e, \text{bn})$$

4. **Kasus ketika nilai variabel belum diketahui:**

Jika `Y` tidak terdapat dalam evidence, maka dilakukan penjumlahan terhadap semua kemungkinan nilai `y` (True dan False) dari variabel tersebut. Untuk setiap nilai `y` , dihitung:

$$P(Y = y \mid Parents(Y)) \times \text{enumerate_all}(\text{rest}, e + \{Y = y\}, \text{bn})$$

Penjumlahan ini mencerminkan proses **marginalisasi**, yaitu menjumlahkan semua probabilitas dari variabel tersembunyi.

Secara keseluruhan, fungsi ini merepresentasikan langkah matematis berikut:

$$P(e) = \sum_{Y_1, Y_2, \dots, Y_n} \prod_i P(Y_i \mid Parents(Y_i))$$

dengan syarat bahwa setiap parent harus sudah didefinisikan sebelum child-nya (topological order).

Fungsi **enumerate_all** merupakan fondasi utama dari algoritma enumerasi pada Bayesian Network dan digunakan oleh fungsi **enumeration_ask()** untuk menghitung distribusi probabilitas bersyarat. Melalui pendekatan rekursif ini, kita dapat menghitung probabilitas total dari sebuah konfigurasi bukti secara sistematis dengan tetap menjaga keakuratan hasil.

```
In [13]: psource(enumeration_ask)
```

```

def enumeration_ask(X, e, bn):
    """
    [Figure 14.9]
    Return the conditional probability distribution of variable X
    given evidence e, from BayesNet bn.
    >>> enumeration_ask('Burglary', dict(JohnCalls=T, MaryCalls=T), burglary
    ... ).show_approx()
    'False: 0.716, True: 0.284'"""
    assert X not in e, "Query variable must be distinct from evidence"
    Q = ProbDist(X)
    for xi in bn.variable_values(X):
        Q[xi] = enumerate_all(bn.variables, extend(e, X, xi), bn)
    return Q.normalize()

```

Fungsi **enumeration_ask(X, e, bn)** digunakan untuk melakukan **inferensi eksak (exact inference)** pada jaringan Bayesian menggunakan metode **enumerasi**. Fungsi ini menghitung distribusi probabilitas bersyarat dari variabel kueri **X**, berdasarkan bukti (*evidence*) yang diberikan **e**, pada jaringan Bayesian **bn**.

Fungsi ini merupakan implementasi dari algoritma yang dijelaskan dalam **Figure 14.9** buku *Artificial Intelligence: A Modern Approach (AIMA)*, di mana inferensi dilakukan dengan menjumlahkan semua kemungkinan nilai dari variabel tersembunyi (*hidden variables*) dan menormalkan hasilnya agar membentuk distribusi probabilitas yang valid.

Parameter fungsi:

- **X** → variabel acak yang ingin dihitung distribusi probabilitasnya.
- **e** → dictionary yang berisi bukti dalam format `{variable: value}`.
- **bn** → objek jaringan Bayesian (`BayesNet`) yang digunakan untuk inferensi.

Langkah-langkah kerja fungsi:

1. Validasi variabel kueri:

Fungsi memastikan bahwa variabel kueri **X** tidak termasuk dalam evidence, karena kita ingin menghitung probabilitasnya, bukan menggunakannya sebagai kondisi.

```
assert X not in e
```

2. Inisialisasi distribusi hasil: Dibuat objek distribusi probabilitas kosong `Q = ProbDist(X)` yang akan menyimpan hasil berupa nilai probabilitas

$$P(X = \text{True} \mid e) \quad \text{dan} \quad P(X = \text{False} \mid e)$$

3. Enumerasi semua kemungkinan nilai X: Untuk setiap kemungkinan nilai x_i dari variabel X (True dan False), fungsi menghitung probabilitas total dengan:

```
Q[x_i] = enumerate_all(bn.variables, extend(e, X, x_i), bn)
```


Artinya, untuk setiap nilai X , kita menghitung probabilitas gabungan dari seluruh variabel dalam jaringan yang konsisten dengan bukti e dan kondisi $X = x_i$, menggunakan fungsi `enumerate_all`.

4. **Normalisasi hasil:** Setelah semua nilai probabilitas dihitung, hasilnya dinormalisasi agar totalnya bernilai 1 dengan:

```
return Q.normalize()
```

Proses normalisasi memastikan bahwa distribusi probabilitas yang dihasilkan valid, sesuai dengan:

$$P(X = \text{True} \mid e) + P(X = \text{False} \mid e) = 1$$

Dengan demikian, fungsi `enumeration_ask()` berperan sebagai *driver function* utama dalam inferensi eksak menggunakan enumerasi pada Bayesian Network.

Sekarang kita akan menghitung probabilitas **$P(\text{Burglary}=\text{True} \mid \text{JohnCalls}=\text{True}, \text{MaryCalls}=\text{True})$** menggunakan jaringan Bayesian **burglary** yang telah dibuat sebelumnya. Proses ini dilakukan dengan memanfaatkan fungsi `enumeration_ask()`, yang akan menghitung distribusi probabilitas bersyarat berdasarkan bukti yang diberikan.

Fungsi `enumeration_ask` menerima tiga argumen utama:

- **X** → variabel yang ingin diketahui probabilitasnya (query variable).
- **e** → bukti (*evidence*) dalam bentuk dictionary, misalnya `{'JohnCalls': True, 'MaryCalls': True}`.
- **bn** → objek jaringan Bayesian yang digunakan untuk melakukan inferensi.

Dengan kata lain, fungsi ini mencari nilai dari:

$$P(X \mid e) = \alpha P(X, e)$$

di mana:

- **X** adalah variabel kueri (misalnya *Burglary*),
- **e** adalah bukti (misalnya *JohnCalls=True, MaryCalls=True*), dan
- **α** adalah faktor normalisasi agar total probabilitas bernilai 1.

Berikut contoh penerapan fungsi pada jaringan **burglary**:

```
In [18]: ans_dist = enumeration_ask('Burglary', {'JohnCalls': True, 'MaryCalls': True}, b
print(ans_dist[True]) # P(Burglary=True | JohnCalls=True, MaryCalls=True)
print(ans_dist[False]) # P(Burglary=False | JohnCalls=True, MaryCalls=True)
```

```
0.2841718353643929
```

```
0.7158281646356071
```

Hasilnya adalah:

```
'False: 0.716, True: 0.284'
```

Artinya, ketika John dan Mary sama-sama menelepon, probabilitas bahwa terjadi pencurian adalah:

$$P(\text{Burglary} = \text{True} \mid \text{JohnCalls} = \text{True}, \text{MaryCalls} = \text{True}) = 0.284$$

Sedangkan probabilitas tidak terjadi pencurian adalah:

$$P(\text{Burglary} = \text{False} \mid \text{JohnCalls} = \text{True}, \text{MaryCalls} = \text{True}) = 0.716$$

Dengan demikian, fungsi **enumeration_ask()** berhasil melakukan inferensi probabilistik secara eksak berdasarkan struktur dependensi yang terdapat pada jaringan Bayesian. Hasil ini juga memperlihatkan bagaimana informasi dari *evidence nodes* (JohnCalls dan MaryCalls) dapat memperbarui keyakinan terhadap *cause node* (Burglary) menggunakan prinsip *Bayesian inference*.

Kesimpulan

Pada praktikum ini, kita telah mempelajari bagaimana proses **inferensi probabilistik** dilakukan menggunakan **Bayesian Network** dengan metode **enumerasi** (*exact inference*). Bayesian Network memungkinkan representasi yang efisien dari **full joint probability distribution** dengan cara memanfaatkan hubungan sebab-akibat antarvariabel dalam bentuk graf berarah tanpa siklus (*Directed Acyclic Graph – DAG*).

Dari hasil implementasi, dapat disimpulkan bahwa:

1. **Bayesian Network** secara signifikan mengurangi kompleksitas perhitungan probabilitas karena hanya perlu menyimpan **Conditional Probability Table (CPT)** untuk setiap variabel, bukan keseluruhan distribusi gabungan.
2. **Inferensi menggunakan enumerasi** dilakukan dengan menghitung semua kemungkinan nilai variabel tersembunyi yang konsisten dengan bukti yang diberikan, kemudian menormalkan hasilnya agar membentuk distribusi probabilitas valid.
3. Fungsi **enumerate_all()** berperan dalam menjumlahkan semua probabilitas sesuai struktur jaringan, sedangkan **enumeration_ask()** digunakan untuk memperoleh probabilitas bersyarat dari variabel kueri berdasarkan evidence.
4. Hasil perhitungan inferensi menunjukkan bahwa bukti (seperti *JohnCalls* dan *MaryCalls*) dapat secara langsung memperbarui kepercayaan terhadap variabel penyebab (*Burglary*) sesuai prinsip **Bayes' theorem**.

Sebagai contoh, hasil inferensi:

$$P(\text{Burglary} = \text{True} \mid \text{JohnCalls} = \text{True}, \text{MaryCalls} = \text{True}) = 0.284$$

menunjukkan bahwa probabilitas pencurian meningkat dari prior $P(\text{Burglary}) = 0.001$ menjadi 0.284 setelah adanya bukti bahwa John dan Mary menelepon.

Hal ini membuktikan bahwa **Bayesian Network** merupakan alat yang kuat untuk merepresentasikan dan menalar ketidakpastian dalam sistem cerdas, dengan dasar teori probabilitas yang konsisten dan dapat dihitung secara sistematis melalui enumerasi.

